PROGRAMA · TRAIN THE TRAINERS

Formación *AI Engineering.*

Prompting, contexto, seguridad, agent skills, templates del SDLC, tool calling con MCP, optimización y casos E2E (Hackathon)

DURACIÓN	FORMATO	COBERTURA
8 módulos × 5 h · 40 h	**Trainers internos BCP**	**5 grupos × 5 personas**

© 2026 NTT DATA, Inc.

Recorrido completo — Vista del programa

Ruta de aprendizaje, *5 días.*

Cinco jornadas de 09:00 a 18:00. Examen de entrada el Día 1; Gate G1 y Gate G2 al cierre de los Días 2 y 4.

1 Día

Prompting & Secure AI Engineering

- Prompting + Context Engineering
- Secure AI Engineering
- Optimización + Refactor
- Docs + Unit Testing

EXAMEN DE ENTRADA
Día 1 · 09:30–10:00

2 Día

Agent Skills, Templates & MCP

- Anatomía del Agent Skill
- Templates del nuevo SDLC
- Tool Calling
- MCP aplicado al SDLC

GATE G1 · EVALUACIÓN
Día 2 · 17:30–18:00

3 Día

SDD + Agentes

- OpenSpec, Spec Kit y Constitution
- Specify → Plan → Tasks → Validate
- Fundamentos de agentes
- Routing, Evaluator y HITL

4 Día

Agentic Engineering in Practice

- CrewAI: agentes por roles
- LangGraph: state y routing
- Harness Engineering
- Guardrails

GATE G2 · EVALUACIÓN
Día 4 · 17:30–18:00

5 Día

Hackathon AI Engineering

- Alinear y priorizar
- Construir en dos sprints
- Validar y congelar la versión
- Demo Arena y certificación

✓ CERTIFICACIÓN
Día 5 · 17:30–18:00

© 2026 NTT DATA, Inc.

Programa — Horario por jornada

Cinco jornadas, *un mismo ritmo.*

	DÍA 1 Prompting & Secure AI	DÍA 2 Skills, Templates & MCP	DÍA 3 SDD + Agentes	DÍA 4 Agentic Engineering	DÍA 5 Hackathon E2E
08:45	08:45–09:00 Coffee break	08:45–09:00 Coffee break	08:45–09:00 Coffee break	08:45–09:00 Coffee break	08:45–09:00 Coffee break y registro
09:00	09:00–09:15 Introducción BCP	09:00–09:15 Apertura y repaso Día 1	09:00–09:15 Apertura y repaso Día 2	09:00–09:15 Apertura y repaso Día 3	09:00–09:15 Kick-off: reglas y reto
	09:15–09:45 **Examen de entrada**	09:15–10:00 S1 · Anatomía del Agent Skill	09:15–10:00 S1 · SDD: OpenSpec, Spec Kit	09:15–10:00 S1 · CrewAI: agentes por roles	09:15–09:45 1 · Alinear
10:00	09:45–10:30 M1 · Prompting + Context Eng.	10:00–10:50 Lab 1 · Skill básico versionado	10:00–10:50 Lab 1 · Definición técnica + contrato	10:00–11:00 Lab 1 · Crew API / BE / FE / reviewer	09:45–10:15 2 · Priorizar
	10:30–11:15 Lab 1 · Reescribir pedidos vagos	10:50–11:30 S2 · Templates del SDLC	10:50–11:30 S2 · Specify → Plan → Tasks	11:00–11:35 S2 · LangGraph: state y routing	10:15–10:30 Checkpoint 1 · Scope
11:00	11:15–11:50 M2 · Secure AI Engineering	11:30–12:30 Lab 2 · Skill desde template	11:30–12:30 Lab 2 · Spec-to-tasks	11:35–12:30 Lab 2 · Graph con estado	10:30–12:30 3 · Construir · Sprint 1
12:00	11:50–12:30 Lab 2 · Detección y corrección segura				
13:00	12:30–13:30 Almuerzo	12:30–13:30 Almuerzo	12:30–13:30 Almuerzo	12:30–13:30 Almuerzo	12:30–13:30 Almuerzo
14:00	13:30–14:15 M3 · Refactor y optimización + demo	13:30–14:20 S3 · Tool calling + demo	13:30–14:15 S3 · Fundamentos de agentes	13:30–14:00 S3 · Harness Engineering	13:30–14:45 3 · Construir · Sprint 2
	14:15–15:20 Lab 3 · Refactor guiado con Copilot	14:20–15:20 S4 · MCP: tools, resources y permisos	14:15–15:25 Lab 3 · Single-agent	14:00–15:15 Lab 3 · Harness del workflow	14:45–15:30 4 · Validar
15:00	15:20–16:00 M4 · Docs + unit testing asistido	15:20–16:00 Lab 3 · Skill + MCP en local	15:25–16:00 S4 · Routing, Evaluator y HITL	15:15–16:00 S4 · Guardrails	15:30–16:00 Checkpoint 2 · Freeze
16:00	16:00–16:15 Coffee break	16:00–16:15 Coffee break	16:00–16:15 Coffee break	16:00–16:15 Coffee break	16:00–16:15 Coffee break
	16:15–17:45 Lab 4 · Docs y unit tests	16:15–17:15 Lab 3 (cont.) · Skill + MCP	16:15–17:45 Lab 4 · Agente + review gate	16:15–17:10 Lab 4 · Agente seguro	16:15–17:15 5 · Demo Arena · 2 salas × 5 equipos
17:00		17:15–17:30 Cierre del Día 2		17:10–17:30 Síntesis · puente al Día 5	17:15–17:30 Deliberación del jurado
18:00	17:45–18:00 Cierre · puente al Día 2	17:30–18:00 **Gate G1 · Evaluación**	17:45–18:00 Cierre · puente al Día 4	17:30–18:00 **Gate G2 · Evaluación**	17:30–18:00 **Cierre + Certificado**

© 2026 NTT DATA, Inc.

Contenido · Laboratorio · Apertura · cierre · Examen y Gates G1 / G2 · Certificación · Coffee · almuerzo

DÍA 2 · FORMACIÓN IA

Agent Skills, Templates, Tool Calling & MCP.

Diseñar capacidades reutilizables, especializar el SDLC, invocar herramientas con criterio y abrir capacidades externas de forma gobernada. Cuatro paradas para llevar lo visto el Día 1 a contratos, plantillas y servidores reales.



AUDIENCIA

Equipos de desarrollo y arquitectura de BCP que completaron el Día 1.

FORMATO

4 sesiones teórico-prácticas.

PROPÓSITO

Salir con un catálogo inicial de Skills, una plantilla trazable y un servidor MCP mínimo corriendo en local.

© 2026 NTT DATA, Inc.

AGENTS SKILLS.

El nuevo estándar para construir capacidades agénticas en el SDLC



Te imaginas un futuro donde podamos incluir capacidades adicionales que no tenemos y que podemos disponibilizar?.

Los Skills funcionan habilitando estas nuevas capacidades a los agentes para que puedan emplearlas en tareas específicas.

© 2026 NTT DATA, Inc.

Puente entre lo visto y lo nuevo **Por qué este día importa**

Del prototipo *a un sistema* que se puede *mantener*.

El Día 1 abrió el terreno: *prompting*, *razonamiento* y *Riesgos de IA*. El Día 2 convierte ese terreno en capacidades reutilizables, plantillas trazables, invocaciones gobernadas y un protocolo estándar para abrir datos y herramientas externas.

AL TERMINAR EL DÍA PODRÁS

01
Diseñar Agent Skills reutilizables con entradas, salidas, límites y validación explícitos.

02
Especializar el SDLC en seis plantillas trazables para API, backend, frontend, QA, docs y review.

03
Decidir cuándo invocar tool calling frente a prompt libre y orquestar varios patrones.

04
Integrar MCP bajo mínimo privilegio, auditoría y separación de capacidades en el SDLC.

VALOR NTT DATA

Terminas con un caso end-to-end: Un Skill de revisión de catálogo, una plantilla de API, una tool con invocación tipada y un servidor MCP mínimo publicando dos capacidades — corriendo en local — y todo conectado por un identificador de trazabilidad común.

© 2026 NTT DATA, Inc.

Recorrido del día 2 Cuatro paradas

Agenda.

Cada sesión tiene introducción, anatomía, ejemplo y criterios. El cierre comparte identificadores y formato de evidencia para que las cuatro paradas se conecten entre sí.

01 Agent Skills

Definir, escribir, versionar y gobernar capacidades reutilizables para el catálogo del equipo.

02 Templates SDLC

Seis plantillas trazables que especializan el ciclo: API, backend, frontend, QA, docs y review.

03 Tool Calling

Patrones de invocación, herramientas tipadas, errores, reintentos e idempotencia.

04 MCP

Conectar el modelo con datos y herramientas externas bajo mínimo privilegio y auditoría.

05 Laboratorios aplicados

Cruzar las cuatro paradas con un identificador de trazabilidad común, evidencia ejecutable y criterios de cierre verificables.

06 GATE 1

Evaluación de sesiónes dia 1 y 2: prompting y agent skills.

META DE LA JORNADA

Salir con un vocabulario común, ejemplos vivos y criterios para decidir **cuándo** crear un Skill, **cuándo** una plantilla, **cuándo** llamar a una tool y **cuándo** abrir un servidor MCP — todo conectado por el mismo identificador de trazabilidad.

© 2026 NTT DATA, Inc.

PARADA 01 · Sesión 1 de 4

01

Anatomía del *Agent Skill*.

Definir, escribir, versionar y gobernar la unidad mínima de capacidad que el equipo va a invocar una y otra vez.

© 2026 NTT DATA, Inc.

Qué es (y qué no) un Agent Skill.

CITA DE TRABAJO

Un Agent Skill es una unidad de conocimiento y comportamiento reutilizable.
Puede describir una tarea, una política, una forma de razonar, o un procedimiento de generación y validación.



UN SKILL NO ES

✗ una instrucción aislada en una conversación
✗ una garantía de ejecución correcta
✗ un sustituto de permisos, pruebas o revisión humana
✗ una especificación completa del producto

UN SKILL SÍ HACE EXPLÍCITO

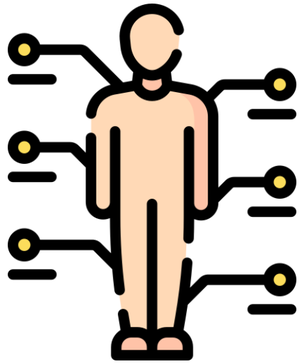
- Cuándo se usa y cuándo no
- Qué información necesita y qué resultado Entrega
- Qué límites debe respetar y cómo se valida
- Qué evidencia debe conservarse para auditar

PRINCIPIO DE DISEÑO

Un Skill es un **contrato**, no una receta. Si el contrato es claro, el equipo sabe cuándo invocarlo, qué esperar y cómo auditarlo. Sin contrato, todo eslóganes.

© 2026 NTT DATA, Inc. · BCP · Día 2

El contrato *antes* del comportamiento.



¿Qué responde cada parte del contrato?

Antes de generar nada, el Skill declara tres cosas: quién es, qué necesita y qué devuelve.

BLOQUE 01

Metadata

Identifica la capacidad y su contexto de uso:

- Nombre e ID estable
- Versión y propósito
- Audiencia y condiciones de activación
- Xasos de no uso e inputs sensibles

BLOQUE 02

Inputs

Describe la información mínima y distingue:

- Datos confirmados vs. opcionales
- Supuestos y preguntas abiertas
- Datos prohibidos o sensibles

Si falta información crítica, el Skill debe detenerse o preguntar, no inventar.

BLOQUE 03

Output

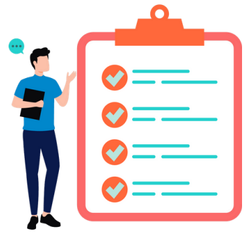
Define la forma exacta de la respuesta:

- Secciones y campos estructurados
- formato e idioma
- Longitud y nivel de detalle

Output sin estructura obliga al revisor a interpretar — convierte cada revisión en una negociación.

© 2026 NTT DATA, Inc. · S1 de 4 · BCP

Lo que el Skill *no* puede hacer y la prueba de que *sí lo hizo*.



Tres bloques convierten el contrato en algo verificable: **Constraints** dicen qué *no* se puede, **Validation** convierte el contrato en criterios observables y **Evidence** deja rastro de la ejecución.

BLOQUE 04

Constraints

Establecen límites de seguridad y calidad:

- no inventar requisitos
- no incluir secretos ni datos PII
- no cambiar archivos fuera de alcance
- respetar stack y convenciones

BLOQUE 05

Validation Rules

Convierten el contrato en criterios observables:

- estructura del artefacto
- contenido y completitud
- calidad técnica
- seguridad aplicada

BLOQUE 06

Evidence

Lo que debe quedar disponible para auditar:

- versión del Skill utilizada
- entradas y salidas completas
- comandos ejecutados
- resultados de pruebas

REGLA OPERATIVA

Sin evidencia, no se sabe si el Skill hizo algo mal — solo se sospecha. Tres capas mínimas por defecto: **logs de entrada y salida**, **resultado de la validación** y **hash de la versión**.

© 2026 NTT DATA, Inc. · S1 de 4 · BCP

Un Skill no *ejecuta*: primero *revisa* y *pregunta*.

> *«Una organización quiere importar un catálogo de libros desde archivos CSV antes de cargarlo a su sistema.»*

SKILL

`catalog-import-review`

LO QUE EL SKILL HACE

Comportamiento esperado

› Recibe estructura del archivo, volumen, campos obligatorios y reglas de negocio.
› Genera un resumen de alcance, riesgos, validaciones y preguntas abiertas.
✗ Rechaza archivos que contengan datos personales no autorizados.
› Pide una muestra de entrada y una muestra de salida esperada.
› Exige evidencia de registros aceptados, rechazados y motivos.

POR QUÉ FUNCIONA

Análisis antes que ejecución

El Skill no modifica la base de datos. Solo revisa. Separar ***análisis*** de ***ejecución*** reduce el riesgo de efectos secundarios y deja un punto claro de revisión humana entre medio.

VARIACIÓN NATURAL

¿Qué cambiarías para una biblioteca pública?

Añadir Constraints sobre copyrights, ampliar rechazo de URLs externas, exigir anonimización del ISBN privado, y un Validation que distinga ***duplicados legítimos*** vs. ***duplicados sospechosos***.

Concepto del ejemplo basado en el PDF BCP v2 · Sección 2: Agent Skills

© 2026 NTT DATA, Inc. · S1 de 4 · BCP

Un Skill *no es eterno*: pasa por cuatro estado.

Un mismo Skill puede tener varias versiones vivas durante una ventana de convivencia, pero el recorrido del contrato es lineal: se diseña, se publica, se deprecia y se archiva.



© 2026 NTT DATA, Inc. · S1 de 4 · BCP

Tres tipos de cambio, una *regla* de inmutabilidad.

MAJOR · BREAKING

v2.0

Cambios incompatibles: se eliminan parámetros, se alteran esquemas JSON o se redefine el propósito.

MINOR · ADITIVO

v1.1

Cambios aditivos: nuevos parámetros opcionales o datos adicionales en la salida del Skill.

PATCH · SEGURO

v1.0.1

Optimizaciones de prompts y corrección de bugs sin alterar el contrato ni la salida.

REGLA DE INMUTABILIDAD

Nunca sobreescribas una versión publicada de un Skill en producción. Si lanzas **v1.2.0**, ese tag o snapshot debe **permanecer idéntico** mientras conviva con versiones nuevas.

EJEMPLO

catalog-import-review

```
v1.0.0 — contrato inicial
v1.1.0 — agrega campo idioma_destino opcional
v2.0.0 — cambia required_inputs (breaking)
```

CONSEJO OPERATIVO

Producción siempre lee una versión exacta

Apunta tu cliente a ***skill_id@v1.1.0***, nunca a *latest*. Los aliases abiertos rompen la trazabilidad y borran evidencia histórica.

© 2026 NTT DATA, Inc. · S1 de 4 · BCP

Primero la *medición*, luego la iteración.

LA INVERSIÓN

TDD también existe para Skills

En lugar de construir y luego probar, defines **cómo se evaluará** el comportamiento esperado. Esa evaluación guía las iteraciones sobre el prompt, el modelo o el agente.

DATO NARRADO EN VOZ ALTA

80 / 20

Antes de tocar un prompt, ***80%*** del tiempo se va en definir lo aceptable. El ***20%*** que queda optimiza mucho más rápido.

PIEZA 01

Gold Standard

Conjunto de casos de prueba con resultados esperados. Mide el no-determinismo del Skill y compara.

Mínimo 20 casos.

PIEZA 02

Regresión

Validación automatizada de precisión y formato ante cualquier cambio de versión. La red de seguridad del catálogo.

No se publica sin pase de regresión.

PIEZA 03

LLM-as-Judge

Un modelo supervisor verifica tono, coherencia y cumplimiento de políticas cuando la métrica humana no escala.

Úsalo como capa, no como veredicto.

PIEZA 04

Inferencia humana

Muestreo manual paraSkills críticos. Una persona mira 20 casos por release y firma la promotion.

No negociable para riesgo medio.

© 2026 NTT DATA, Inc. · S1 de 4 · BCP

Depreciar *sin sorprender* y dejar *registro* de todo.

BLOQUE A · ESTRATEGIA DE SUNSET

No apagues una versión de golpe

› **Sunset window** · periodo explícito de convivencia entre la versión antigua y la nueva, normalmente 3–6 meses.› **In-context warning** · metadatos en el prompt indican que la herramienta está descontinuada, pero sigue respondiendo.› **Aliases controlados** · en producción se apunta a versiones exactas, no a `latest`.

BLOQUE B · TELEMETRÍA Y AUDITORÍA

Lo que no se ve no se mejora

› **Stamping en logs** · cada invocación registra `skill_id` + `version` exactos.› **Lineage & Git SHA** · trazabilidad del commit exacto que originó la versión desplegada.› **Sovereignty & drift** · detección temprana de variaciones de comportamiento inesperadas.

RELACIÓN QUE NO DEBE ROMPERSE

La **depreciación** y la **telemetría** son la misma conversación: si no puedes demostrar cuándo y cómo se retiró una versión, tampoco puedes explicar por qué algo dejó de funcionar.

© 2026 NTT DATA, Inc. · S1 de 4 · BCP

No todo merece un Skill: una *decisión*, cuatro *señales*.

CREAR UN SKILL CUANDO

La situación se repite o se audita

01 Se repite con frecuencia — no es un caso único.
02 Tiene reglas o un formato estable que conviene fijar.
03 El costo de inconsistencia es alto y se nota.
04 Se requiere auditarlo o comparar resultados entre equipos.

Regla práctica: si necesitas repetir la misma dirección más de tres veces, vale la pena invertir una hora en definir el Skill.

MEJOR PROMPT LIBRE CUANDO

La exploración todavía no merece estructura

01 Es exploratorio, sin reglas todavóa bien definidas.
02 Es un caso único que no merece ser versionado.

Forzar estructura antes de entender el problema solo agrega rigidez y términos rimbombantes.



© 2026 NTT DATA, Inc. · S1 de 4 · BCP

Promover un Skill *es* un compromiso, *no un sello*.

FICHA MÍNIMA DE CATÁLOGO

Lo que un catálogo necesita saber

- nombre e identificador único
- versión vigente y Owner asignado
- nivel de riesgo: bajo / medio / alto
- estado de certificación
- fecha de última revisión

Si la ficha no se puede llenar, el Skill no está listo para el catálogo.

CRITERIOS PARA PROMOVER

El Skill entra si…

- más de un equipo o rol lo usará
- toca artefactos que van a producción
- fue probado con casos incompletos y maliciosos
- tiene un dueño que acepta mantenerlo
- genera evidencia auditabe

SEÑALES DE QUE TODAVÍA NO

Espera antes de promover

- nadie quiere ser el dueño formal
- solo se ha probado con el camino feliz
- duplica uno existente — mejor consolidar
- cambia tan constante que versionarlo no aporta

CONSEJO PARA EMPEZAR

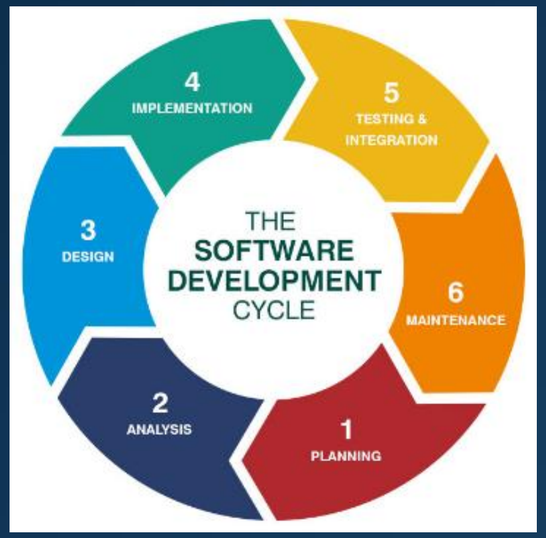
Tres Skills propios del equipo, bien mantenidos, enseñan más que veinte **a medias**. La promesa del catálogo no es la cantidad, sino la confiabilidad.

© 2026 NTT DATA, Inc. · S1 de 4 · BCP

PARADA 02 · Sesión 2 de 4

02

Templates para el ciclo *SDLC*.



Plantillas trazables que especializan una fase, comparten origen y no rompen la trazabilidad entre sí.

© 2026 NTT DATA, Inc.

Un Template SDLC es un Skill que *aterriza* una *fase* concreta del desarrollo.

Un Template convierte una ***intención*** en un artefacto propio de una fase del SDLC. Cada Template se comporta como un Skill especializado: tiene su propósito, sus entradas, su salida y sus criterios — pero todos ***comparten un mismo origen*** (el requerimiento) y deben mantenerse trazables entre sí.

LOS SEIS TEMPLATES DEL CICLO

TEMPLATE 01

API

Contrato de servicio. Esquemas, endpoints y catálogo de errores.

TEMPLATE 02

Backend

Diseño e implementación. Lógica, persistencia y seguridad.

TEMPLATE 03

Frontend

Estados, interacción y accesibilidad mapeada al contrato.

TEMPLATE 04

Test

Matriz requisito, escenario, prueba y evidencia real.

TEMPLATE 05

Docs

Decisiones, límites conocidos y guía operativa.

TEMPLATE 06

Review

Hallazgos, riesgos residuales y decisión final del cambio.

PRINCIPIO

La **especialización** no debe romper la **trazabilidad**. Todos los artefactos deben conservar un identificador común, alcance, criterios de aceptación y preguntas abiertas.

© 2026 NTT DATA, Inc. · S2 de 4 · BCP

Contrato y tablero *en una sola mirada*.

CONTRATO MÍNIMO

Siete campos por Template

- propósito y fase del SDLC
- entradas requeridas
- artefacto de salida
- supuestos permitidos
- restricciones técnicas y de seguridad
- validaciones y evidencia esperada
- responsable de revisión

Sin alguno de estos siete, el Template queda como buena intención.

TEMPLATE	FASE	ENTRADA CLAVE	SALIDA CLAVE
API	Contrato	Historia de usuario y convenciones de la organización	Especificación de endpoints, esquemas, catálogo de errores
Backend	Diseño e implementación	Contrato de API y reglas de negocio	Diseño de lógica, modelo de datos y plan de seguridad
Frontend	Diseño e implementación	Contrato de API y criterios de aceptación	Catálogo de estados de UI, mapeo estado-componente
Test	Validación	Criterios de aceptación, diseños de BE y FE	Matriz requisito — escenario — prueba — evidencia
Docs · Review	Registro · Decisión	Decisiones de fases previas · Artefactos de todas las fases anteriores	

Salida: *decisiones registradas, límites conocidos, runbook, hallazgos, decisión final del cambio.*

© 2026 NTT DATA, Inc. · S2 de 4 · BCP

El contrato *antes* del código; la lógica *antes* de la persistencia.

TEMPLATE 01

Template de API

«Convertir un requerimiento en un contrato completo y consistente antes de escribir código de backend.»

CUÁNDO SE USA

Al definir o modificar un endpoint, antes de tocar el backend.

ENTRADAS

Historia de usuario · criterios de aceptación · convenciones de la organización · catálogo de errores estándar · dependencias.

SALIDA

Endpoints con métodos y parámetros · esquemas request/response · catálogo de errores con códigos y mensajes · reglas de versionado.

CONDICIÓN DE «PASSED»

Cada endpoint con caso de éxito y caso de error · esquemas válidos (OpenAPI o JSON Schema) · breaking changes marcados.

TEMPLATE 02

Template de Backend

«Traducir el contrato y las reglas de negocio en un diseño de implementación seguro y consistente.»

CUÁNDO SE USA

Cuando el contrato de API ya existe, antes de persistencia y lógica.

ENTRADAS

Contrato de API · reglas de negocio · restricciones de stack · requisitos no funcionales.

SALIDA

Diseño de la lógica y validaciones · modelo de datos y migraciones · estrategia de seguridad y transacciones.

CONDICIÓN DE «PASSED»

Cubre todos los endpoints · reglas críticas explícitas · datos sensibles identificados y tratados.

© 2026 NTT DATA, Inc. · S2 de 4 · BCP

Estados *antes* de componentes; pruebas *antes* de cerrar el ticket.

TEMPLATE 03

Template de Frontend

«Definir los estados de la interfaz y la experiencia para cubrir el contrato y los criterios.»

CUÁNDO SE USA

En paralelo con el contrato de API, antes de implementar UI.

ENTRADAS

Contrato de API · criterios funcionales · guía de estilo o design system · requisitos de accesibilidad.

SALIDA

Catálogo de estados de UI · mapeo estado → componente · reglas de accesibilidad y mensajes.

CONDICIÓN DE «PASSED»

Todo estado backend tiene su UI · mensajes comprensibles · accesibilidad mínima cumplida.

TEMPLATE 04

Template de Test

«Convertir requisitos y contrato en una estrategia de pruebas ejecutable y trazable.»

CUÁNDO SE USA

En paralelo con diseño de BE/FE, antes de cerrar la funcionalidad.

ENTRADAS

Criterios de aceptación · contrato de API · niveles disponibles: unitario, integración, E2E y seguridad.

SALIDA

Matriz requisito → escenario → prueba → evidencia · estrategia de datos · umbral de cobertura.

CONDICIÓN DE «PASSED»

Cada criterio con al menos un escenario · evidencia real de ejecución · sin datos productivos.

© 2026 NTT DATA, Inc. · S2 de 4 · BCP

Documentar es *decidir*; revisar es *cerrar* con alguien mirando.

TEMPLATE 05

Template de Docs

«Registrar las decisiones, límites y forma de operar la funcionalidad.»

SALIDA

Decisiones con justificación · límites conocidos y deuda aceptada · guía operativa (runbook) · changelog.

CONDICIÓN DE «PASSED»

Toda decisión relevante documentada · límites y supuestos explícitos.

TEMPLATE 06

Template de Review

«Verificar antes de aprobar que alcance, seguridad, contrato y pruebas son consistentes entre sí.»

SALIDA

Hallazgos encontrados · riesgos residuales, aceptados o bloqueantes · decisión final: aprobado, con condiciones o rechazado.

CONDICIÓN DE «PASSED»

Cada hallazgo vinculado a evidencia · decisión sustentada, no por opinión.

CASO FICTICIO

Reserva de salas

Los seis Templates convergen en una sola funcionalidad:

› API: disponibilidad, reserva, cancelación y errores.
› BE: reglas de solapamiento.
› FE: libre, reservada, vencida, error.
› TEST: reglas por escenarios.
› DOCS: decisiones y límites.
› REVIEW: pasado antes de merge.

NO DECIDIR POR LA CANTIDAD DE ARTEFACTOS

Ningún Template debería declarar que la reserva está lista solo porque generó archivos. La decisión depende de **validaciones** y **evidencia** — no del **conteo de artefactos producidos**.

© 2026 NTT DATA, Inc. · S2 de 4 · BCP

Templates *que* funcionan *y* errores *que se repiten*.

BUENAS PRÁCTICAS

Doce hábitos que pagan

- mantenerlos ***pequeños*** y ***componibles***
- separar instrucciones de política y de implementación
- usar campos obligatorios para decisiones de alto riesgo
- proporcionar ***ejemplos válidos y rechazados***
- no ocultar la incertidumbre con valores ambiguos
- hacer que las salidas sean legibles por personas y herramientas
- definir una condición clara de `passed`, `failed` o `not_applicable`
- revisar el Template con los roles que lo usarán
- usar un ***identificador de trazabilidad común*** entre los seis
- hacer explícitos los ***handoffs*** entre roles
- evitar que una fase repita la autoridad de otra
- adjuntar ***evidencia*** junto al artefacto

ERRORES FRECUENTES

Cinco trampas que reaparecen

01 Confundir una plantilla con código listo para producción.

02 Generar artefactos ***incompatibles*** entre disciplinas (un estado de Frontend que no existe en Backend).

03 Duplicar reglas de negocio en varios Templates sin una fuente única de verdad.

04 Declarar ***cobertura*** sin ejecutar pruebas.

05 Convertir una decisión técnica en un ***supuesto no declarado***.

Añadir pasos que no dejan evidencia verificable es el más común — y el más caro de revertir.

© 2026 NTT DATA, Inc. · S2 de 4 · BCP

No son sustitutos: *se combinan*.

CAPA 1 · AGENT SKILL

El qué reutilizable

Contrato de capacidad específica. Define entradas, salidas, límites y validación. Vive en el catálogo corporativo.

CAPA 2 · TEMPLATE SDLC

El dónde del ciclo

Contexto de fase en que se integra. Selecciona Skills vigentes del catálogo. Vive con el requerimiento.

REGLA OPERATIVA

El Template elige Skills vigentes

Un Template certificable debe seleccionar únicamente Skills del catálogo. La trazabilidad corre por el mismo `id` que comparten los seis Templates del cambio.

TIPS OPERATIVOS

› un identificador de trazabilidad común
› handoffs explícitos entre roles
› entradas y salidas con nombres concretos
› evidencia junto al artefacto
› autoridad no se repite entre fases
› Templates pequeños, componibles y auditables

© 2026 NTT DATA, Inc. · S2 de 4 · BCP

03

Tool *Calling*.

Cuando el modelo deja de leer: invocar herramientas tipadas, orquestar varias o esperar formatos. La pieza más concreta del camino de un Skill.

© 2026 NTT DATA, Inc.

Cuando el modelo *elige* llamar a una *herramienta*.

Una tool = una acción externa tipada

El modelo recibe una lista de herramientas con ***nombre***, ***descripción*** y ***parámetros JSON Schema***. Decide cuándo invocarlas, con qué argumentos tipados y con cuál encadenar después.

PIENSA EN ELLO ASÍ

El modelo propone la llamada. La aplicación la ejecuta y le devuelve la respuesta.

ESQUEMA DE LA TOOL — DECLARACIÓN

```
name: "reservar_sala"
description: "Reserva una sala para una fecha
y horario determinados."
parameters: {
type: "object",
properties: {
sala_id: { type: "string" },
inicio: { type: "string",
format: "date-time" },
fin: { type: "string",
format: "date-time" }
},
required: ["sala_id","inicio","fin"]
}
```

RESPUESTA QUE LLEGA DE VUELTA

```
status: "ok"
tool_call_id: "rsv_01HM..."
content: {
reserva_id: "R-91234"
sala: "S-204"
estado: "confirmada"
}
```

Tres campos sobre los que pensar: **nombre**, **descripción**, **parámetros**. Si cambias uno, cambia el contrato.

© 2026 NTT DATA, Inc. · S3 de 4 · BCP

Tres patrones *que aparecen* casi en *toda* invocación real.

PATRÓN 01

Secuencial

Una llamada alimenta la siguiente. Útil cuando el segundo paso necesita el resultado del primero.

```
get_user(uid) → get_orders(uid) → summarize(orders)
```

PATRÓN 02

Paralelo

Varias llamadas independientes que el modelo emite en una sola vuelta. Acumula contexto sin sumar latencia.

```
[get_user(uid), get_orders(uid), get_tickets(uid)]
```

PATRÓN 03

Condicional

El modelo decide si lanza o no una herramienta según el resultado de otra anterior.

«si el monto > 1000, requiere aprobación humana antes de continuar».

PATRÓN 04 · STOP CONDITIONS

Límites y corte

Limitar el número de iteraciones, no permitir más de N llamadas por turno y devolver el control cuando se cumple una condición observable. Sin esto, un agente puede iterar eternamente.

CONSEJO PRÁCTICO

Empieza **secuencial**, pasa a **paralelo** cuando los pasos sean independientes, y reserva **condicional** para cuando haya que **frenar** al modelo.

© 2026 NTT DATA, Inc. · S3 de 4 · BCP

Las tools *fallan*.
El plan debe *asumirlo*.

PRINCIPIO

Idempotencia

La misma llamada con los mismos argumentos debe poder ejecutarse ***más de una vez sin alterar el resultado***. Si no es así, reintentar se vuelve un problema.

REINTENTOS

Backoff corto

Reintentar con backoff exponencial, máximo 2 o 3 intentos. Nunca como solución silenciosa: registrar cada intento en logs.

FALLOS QUE NO SE REINTENTAN

Validación y permisos

No reintentar errores de validación o de permisos — retornan el mismo fallo. Distinguirlos en el código de error.

TIEMPOS Y LÍMITES

Timeouts cortos y explícitos

Define timeout por tool, no globales. Si la tool cuelga, prefieres un fallo claro a una espera silenciosa.

PRESUPUESTO POR TURNO

Limita a ***10–15 llamadas por turno*** y a ***3 reintentos por tool***. Cuando se agotan, devuelve el control al usuario o a un ser humano.

PRINCIPIO DE FRENO

Tres categorías de fallo siempre: **recuperable** (reintentar) · **parcial** (seguir con menos) · **terminal** (devolver control). Sin esa taxonomía el modelo no sabe cuándo debe parar.

© 2026 NTT DATA, Inc. · S3 de 4 · BCP

04

MCP aplicado al *SDLC*.

Un protocolo abierto para conectar modelos con datos y herramientas externas — bajo mínimo privilegio, auditoría y separación de capacidades.

© 2026 NTT DATA, Inc.

Un protocolo *estándar* entre modelos y servidores.

Model Context Protocol es una frontera técnica

Un protocolo abierto para que aplicaciones que usan modelos puedan conversar con servidores que exponen capacidades externas. ***No*** es un modelo, ni un agente, ni una autorización universal.

CONCEPTO

Servidor ***no es*** autorización. **Anunciar una capacidad** no garantiza permiso para ejecutarla. La política de seguridad siempre acompaña al servidor.

LOS TRES ACTORES

ACTOR 01

Host

Coordina la interacción con el modelo. Decide qué entra en el contexto y qué se descarta. Vive dentro de la app que invoca al modelo.

ACTOR 02

Cliente MCP

Mantiene la sesión con uno o varios servidores. Conoce las capacidades publicadas y las solicita por el nombre que el modelo eligió.

ACTOR 03

Servidor MCP

Publica capacidades, valida argumentos, aplica límites y decide qué responde. Es la **frontera técnica**, no una autorización universal.

El host **controla el contexto**; el cliente **mantiene la sesión**; el servidor **publica y aplica límites**.

© 2026 NTT DATA, Inc. · S4 de 4 · BCP

Tres primitivas, una *responsabilidad*: el contexto.

LAS TRES PRIMITIVAS QUE UN SERVIDOR PUEDE PUBLICAR

PRIMITIVA 01

Tool

Acción que el modelo solicita al servidor. Tiene nombre, descripción y argumentos JSON tipados.

«Consultar estado de una entrega»

PRIMITIVA 02

Resource

Información identificable que el servidor ofrece. El modelo puede leerla como contexto, sin invocarla como acción.

«Política de devoluciones vigente»

PRIMITIVA 03

Prompt

Plantilla publicada por el servidor. El host puede incluirla como instrucción lista para el modelo.

«Resumen ejecutivo de incidencias»

CONTROL DEL CONTEXTO — RESPONSABILIDAD DEL HOST

¿Qué información se incorpora? ¿Con qué prioridad? ¿Durante cuánto tiempo? **Resource es la fuente**; **contexto es lo que decide entregar al modelo**. Nunca son sinónimos.

El servidor expone. El cliente solicita. ***El host gobierna qué entra y qué se descarta***.

© 2026 NTT DATA, Inc. · S4 de 4 · BCP

Cómo se mueve la conversación y qué *trade-off* aceptas.

CICLO DE LLAMADA TÍPICO

Cinco pasos, una evidencia

01 ***Establecer*** conexión y anunciar capacidades.

02 ***Consultar*** tools, resources o prompts.

03 ***Solicitar*** operación con argumentos tipados.

04 ***Validar*** respuesta y contextualizar.

05 ***Registrar*** sin exponer secretos.

TRANSPORTE 01

stdio (proceso local)

\+ Sencillo, sin puerto público.

– Aislamiento insuficiente si no se acota bien.

TRANSPORTE 02

HTTP (servidor remoto)

\+ Separación cliente/servidor y operación remota.

– Riesgo de autenticación mal configurada.

REGLA DE SELECCIÓN

¿Cómo elegir?

`stdio` para servidores locales del propio host.

`HTTP` cuando los servidores viven aparte y los demás equipos o proveedores los consumen. No mezcles ambos en el mismo servidor sin separar el ciclo de vida.

© 2026 NTT DATA, Inc. · S4 de 4 · BCP

Skill + MCP: *no compiten, se apoyan*.

RELACIÓN ENTRE LAS DOS

Skill define qué · MCP aporta cómo

Skill: lógica reutilizable, contratos y límites.
MCP: acceso gobernado a herramientas y datos externos.

PREGUNTA RÁPIDA

¿Se va a reutilizar?

Si más de un equipo o ***varios Skills*** necesitan los mismos datos y herramientas externas, MCP paga. Si es un caso único, mejor una integración directa.

SÍ USAR MCP

Cuatro señales a favor

- la capacidad se va a invocar desde ***varios lugares***
- hay que separar a quién pertenece cada capacidad
- se necesita ***auditar y observar*** sin tocar el modelo
- los datos son ***sensibles o cambian*** rápido

MEJOR NO USAR MCP

Cuatro señales en contra

- la integración directa ya ***funciona y es simple***
- el caso es ***cerrado*** y no se va a reutilizar
- la capacidad no requiere ***negociación*** de contexto
- el equipo aún no sabe ***qué va a exponer***

UNA PRUEBA SENCILLA

¿Protocolo o integración?

Si ***compartir*** el acceso sin compartir código es tu objetivo, MCP ayuda. Si lo que quieres es encapsular lógica de negocio, Skill + Template alcanza.

© 2026 NTT DATA, Inc. · S4 de 4 · BCP

Cuatro frentes donde *MCP* ya está *moviendo* dinero.

CASO 01

Ingeniería

Docs versionadas, incidencias, contratos API, pipelines CI/CD.

CASO 02

Operaciones

Métricas agregadas, alertas resumidas, runbooks y estado de servicios.

CASO 03

Conocimiento

Políticas internas, catálogos y clasificación antes de exponer.

CASO 04

Automatización

Diferenciar ***leer***, ***proponer***, ***escribir*** y ***aprobar***.

SEGURIDAD · LO QUE NO SE NEGOCIA

Tratar los ***resources*** como ***datos no confiables***. Separar instrucciones del sistema del contenido recuperado. Anunciar capacidad ***no*** es permiso para ejecutarla — la política viaje con el servidor.

CONSEJOS OPERATIVOS

› empezar con **una sola tool** de lectura› tools específicas con argumentos **tipados**

› respuestas **compactas y paginadas**

› marcar qué es reversible o destructive

› simular credenciales insuficientes

› redactar secretos, tokens y PII en logs

© 2026 NTT DATA, Inc. · S4 de 4 · BCP

DÍA 2 · CIERRE Lo que te llevas

Cuatro paradas, *un sistema* por construir.

S1
Skills propios
Tres al catálogo, versionados y con dueños.

S2
Templates trazables
Los seis funcionando con un mismo identificador.

S3
Tool calling tipado
JSON Schema, errores claros, idempotencia cuando se puede.

S4
Servidor MCP mínimo
Publicando dos capacidades, con auditoría y reversibilidad explícitas.

© 2026 NTT DATA, Inc.